

Current Progress

Main Themes

Priority	TODO	IN PROGRESS	DONE (most recently completed tasks on the top)
HIGH	- Self-Learning on different searching and retrieval algorithms - Agent should return more structured outputs for better validation and control over data (Bug)		LangGraph Multi-Agent based implementation (Router Agent routing to the KG/Vector Store (Tools), and an Evaluator Agent assessing the generated answer, whether to re-invoke for generation, or to return the answer). Aditya Khadkikar Git Branch - Merging vector search with graph-based search (from "Make the vector retrieval in the form of a Langchain chain") - RunnableParallel (for two potential answer derivation approaches, from KG or retrieving context from a vector store) - Complex attributes from UnitDB (more unstructured, and need a different data parsing process) Oscar Stackenland - UnitDB Data Cleaning prior to embedding (extracting the model ID, capabilities, attributes and values from blank nodes) Oscar Stackenland
MEDIUM		Example improvement and prompt engineering	
LOW		- Observe more excluded attribute types from the schema (e.g. rdfs:label, skos:prefLabel) (Blocked since we have limited knowledge of the KG/Ontology structure. Need help from David Dirk)	- Semantic Chunking (found that it is not fully applicable to the case of UnitDB data, as it is already structured) - Make the vector retrieval in the form of a Langchain Chain - Embed either the nodes or the textual parts of our graph - Document creation Aditya Khadkikar - text > (textual information from all blank nodes of a given product model) "text" is converted to embeddings. - metadata > (name of the node <kdu...> or non-human-readable ID, concatenated with the human-readable equiv. name, for retrievability) - Vector Search with filtering via OpenSearch (UnitDB) Commit - Verifying with some example Intent Questions - Experimentation for whether the previous exchanges between Human and Model are retained in memory, and subsequent questions can be asked without providing context again (MessageHistory) Aditya Khadkikar - Incorporating memory in the LLM - ConversationBufferMemory - SPARQLWrapper to send queries to a graph on a remote API / local file + error handling to direct a faulty-generated SPARQL query back to the SparQLQACHain to amend, and re-generate. - Found query_endpoint (serialized URL) where a particular graph of a remote dataset can be retrieved, and schema can be generated. - Custom schema generation (overriding the RDFGraph prompt setup from lan gchain_community graphs) - Identified issues: AnnotationProperty types tend to be missing / excluded from the generated schema, which is solved with this way. Git Branch - Schema conversion to JSON, and explicitly declaring to the LLMChain the node property, object properties and datatype properties, with the aim to improve understandability. Git Branch - Validate whether the query follows the schema, and does not invent new properties/relationships - Reason: Executed SPARQL query does not crash if a property that does not exist is part of the query, but runs and simply gives 0 results, however an error is expected. - Exploring rdf2vec or other embeddings as a context enhancing approach - Examples addition - (currently done for two use-cases: "Optical Indicator" identification and "Power Up" instructions) - Takeaways: When retrieving the nodes, it is also necessary to go through the textual fields of each node, to get further contextual information to answer the user query (embeddings?)